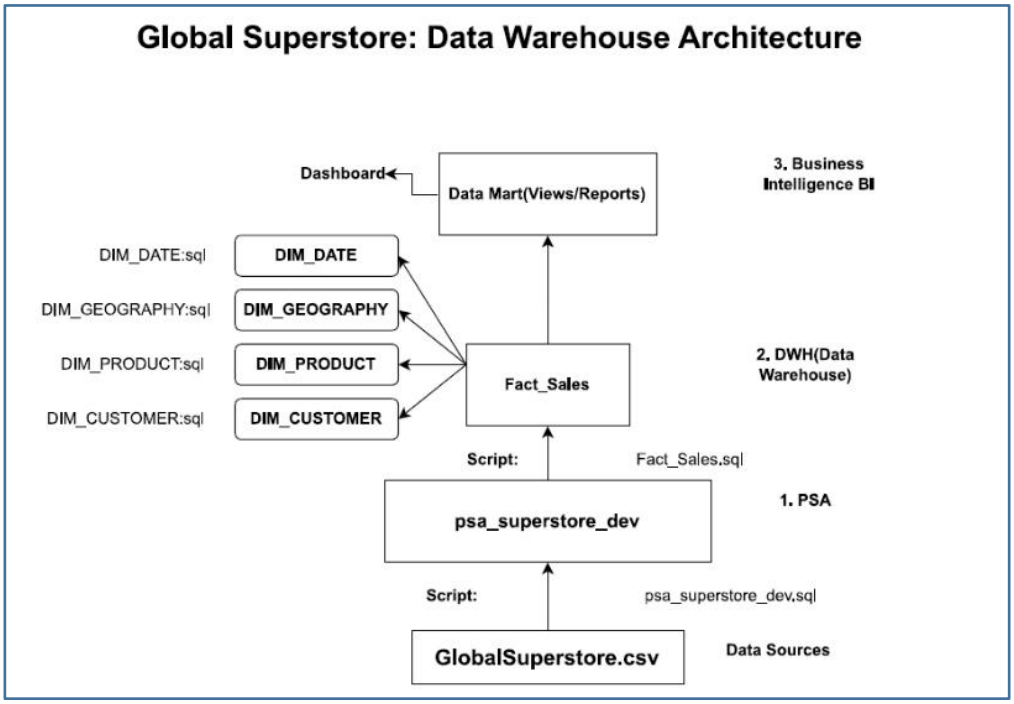
Technical Document (System Overview)

1. Architecture Overview

The system follows a **layered data warehouse architecture** consisting of **Data Source, PSA (Persistent Staging Area), Data Warehouse, and Business Intelligence layers**. This design ensures data consistency, scalability, and clear separation of concerns between ingestion, storage, and analytics.



Pic 1. Data Warehouse Architecture

2. Data Flow Description

2.1 Data Source Layer

Source file: *GlobalSuperstore.csv*

```
df.info()

<class 'pandas.core.frame.DataFrame'>
Index: 51289 entries, 0 to 51289
Data columns (total 28 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Category       51289 non-null  object
 1   City           51289 non-null  object
 2   Country        51289 non-null  object
 3   Customer.ID    51289 non-null  object
 4   Customer.Name  51289 non-null  object
 5   Discount       51289 non-null  float64
 6   Market         51289 non-null  object
 7   Order.Date     51289 non-null  datetime64[ns]
 8   Order.ID       51289 non-null  object
 9   Order.Priority 51289 non-null  object
 10  Product.ID     51289 non-null  object
 11  Product.Name   51289 non-null  object
 12  Profit         51289 non-null  float64
 13  Quantity       51289 non-null  int64
 14  Region         51289 non-null  object
 15  Row.ID         51289 non-null  int64
 16  Sales          51289 non-null  int64
 17  Segment        51289 non-null  object
 18  Ship.Date      51289 non-null  datetime64[ns]
 19  Ship.Mode      51289 non-null  object
 20  Shipping.Cost  51289 non-null  float64
 21  State          51289 non-null  object
 22  Sub.Category   51289 non-null  object
 23  Year           51289 non-null  int64
 24  Market2        51289 non-null  object
 25  weeknum        51289 non-null  int64
 26  Days_to_Ship   51289 non-null  int64
 27  Order_Month    51289 non-null  int32
dtypes: datetime64[ns](2), float64(3), int32(1), int64(6), object(16)
memory usage: 11.2+ MB

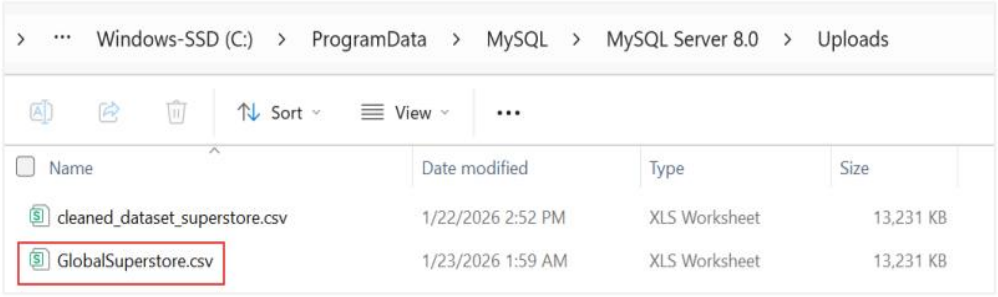
Download cleaned Data

df_clean = df.copy()
df_clean.to_csv("C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/cleaned_dataset_superstore.csv", index=False, date_format="%d/%m/%Y %H:%M")

df_clean.head()
```

Pic 2. Data Source

The source data is obtained from the **Global Superstore dataset**, which is publicly available on Kaggle (https://www.kaggle.com/datasets/fatihilhan/global-superstore-dataset). The raw dataset is first processed in **Python using data preprocessing and data cleansing techniques** to handle missing values, standardize formats, and ensure data consistency. After preprocessing, the cleaned dataset is exported as a CSV file and stored in a designated upload directory (*Pic 1. Data Source*).



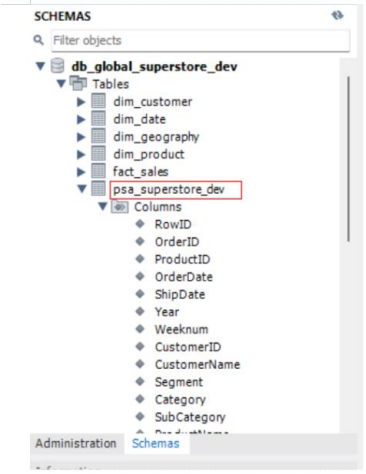
Pic 3. Data Source Folder Path

This cleaned data file is then directly loaded into **MySQL** and stored in the **Persistent Staging Area (PSA)** using bulk load operations.

2.2 PSA (Persistent Staging Area)

Table: *psa_superstore_dev*

This layer serves as a temporary storage for raw data with minimal transformation, ensuring data traceability and enabling validation and reprocessing.




```
LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/GlobalSuperstore.csv"
INTO TABLE psa_superstore_dev
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 LINES
(
  Category,
  City,
  Country,
  CustomerID,
  CustomerName,
  Discount,
  Market,
  OrderDate,
  OrderID,
  OrderPriority,
  ProductID,
  ProductName,
  Profit,
  Quantity,
  Region,
  RowID,
  Sales,
```

The data flow in the **PSA** begins after the cleaned source data is prepared. The cleaned CSV file is placed in the MySQL secure upload directory and ingested into the database using a bulk loading mechanism.

During this step, the `LOAD DATA INFILE` command reads the CSV file and loads the records into the `psa_superstore_dev` table. Explicit column mapping ensures that each field from the source file is correctly aligned with

the PSA table structure, while the header row is ignored to prevent metadata from being loaded as data.

At this stage, the data undergoes **minimal transformation only**. Most attributes are stored using flexible data types to preserve the original values and avoid premature type conversion. This design allows the PSA to function as a reliable checkpoint for data validation, traceability, and error detection.

Once the data is successfully loaded, a primary key constraint on `RowID` guarantees record uniqueness within the PSA. Basic validation steps, such as row count verification and sample data inspection, are performed before the data is passed to the data warehouse layer for further transformation into dimension and fact tables.

2.3 Data Warehouse layer (DWH)

The **Data Warehouse (DWH) layer** is designed to store integrated, cleaned, and structured data optimized for analytical querying and reporting. In this implementation, the data warehouse follows a **star schema architecture**, consisting of multiple dimension tables and a central fact table.

Data is sourced from the Persistent Staging Area (PSA) and transformed into analytical structures through SQL-based ETL scripts. The DWH layer ensures data consistency, referential integrity, and efficient access for downstream analytics.

2.3.1 Dimensional Model Overview

The data warehouse schema consists of the following tables:

Dimension Tables

DIM_DATE
DIM_CUSTOMER
DIM_PRODUCT
DIM_GEOGRAPHY

Fact Table

Fact_Sales

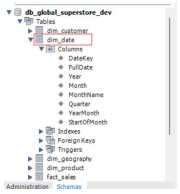
Each dimension table contains descriptive attributes, while the fact table stores quantitative business measures linked to the dimensions through surrogate keys.

2.3.2 Dimensional Model Overview

The data warehouse schema consists of the following tables:

a. Date Dimension (DIM_DATE)

The DIM_DATE table provides a continuous calendar dimension used for time-based analysis.



Key Features:

- Uses a surrogate key (DateKey) in YYYYMMDD format
- **Covers** the full date range derived from OrderDate and ShipDate in PSA
- Enables analysis by year, quarter, month, and month name

b. Customer Dimension (DIM_CUSTOMER)

The DIM_CUSTOMER table stores customer-related attributes.

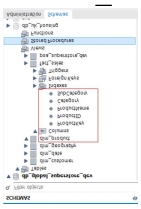


Key Features:

- **Surrogate key**: CustomerKey
- **Business key**: CustomerID
- **Includes** customer name and segment
- Enforces uniqueness on the business key

c. Product Dimension (DIM_PRODUCT)

The DIM_PRODUCT table contains product hierarchy information.

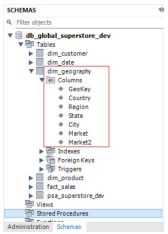


Key Features:

- **Surrogate key**: ProductKey
- **Business key**: ProductID
- Attributes include product name, category, and sub-category
- Uses ON DUPLICATE KEY UPDATE to support idempotent loads

d. Geography Dimension (DIM_GEOGRAPHY)

The DIM_GEOGRAPHY table represents geographic attributes.

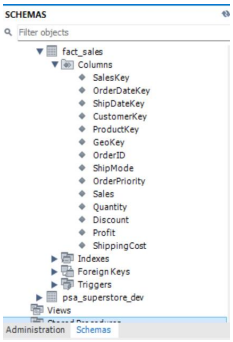


Key Features:

- Surrogate key: GeoKey
- Unique constraint on (Country, Region, State, City)
- Includes market and market classification attributes

2.3.3 Fact Table (Fact_Sales)

The Fact_Sales table stores transactional sales measures and acts as the central table in the star schema.



Measures:

- Sales
- Quantity
- Discount
- Profit
- ShippingCost
- Foreign Keys:
- OrderDateKey, ShipDateKey → DIM_DATE
- CustomerKey → DIM_CUSTOMER
- ProductKey → DIM_PRODUCT
- GeoKey → DIM_GEOGRAPHY

Degenerate Dimensions:

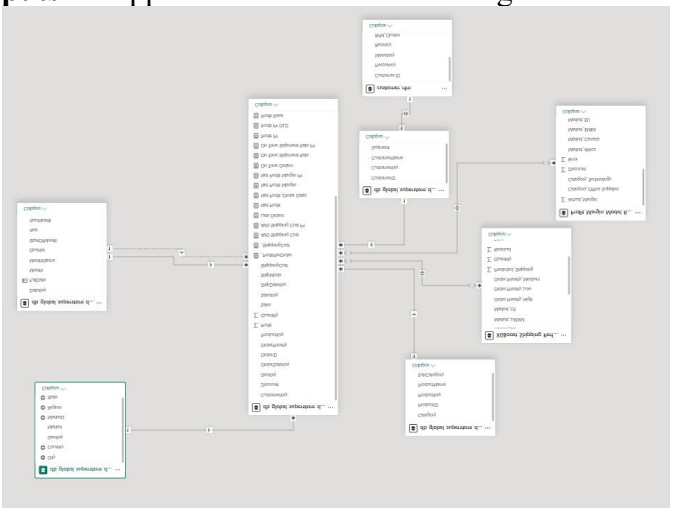
- Ordered
- ShipMode
- OrderPriority

Purpose:

Stores numeric facts used to compute KPIs such as Net Profit, Net Profit Margin, Sales PY, and shipping performance metrics

2.4 Data Mart / BI Layer

At this stage, the data warehouse is extended with **analytical and machine learning outputs** to support advanced business insights in the BI layer.



Power BI Data Model

The Power BI data model is built on top of the star schema from the data warehouse. The Fact_Sales table acts as the central fact table and is connected to dimension tables (DIM_DATE, DIM_PRODUCT, DIM_CUSTOMER, and DIM_GEOGRAPHY) through one-to-many relationships. This structure enables efficient slicing, filtering, and drill-down analysis across time, product, customer, and geography.

In addition to the core warehouse tables, several **analytical result tables** are integrated into the model to enrich reporting capabilities.

2.4.1 Customer RFM Analytics Integration

A customer RFM (Recency, Frequency, Monetary) table is generated using Python to support customer segmentation analysis. The RFM metrics and cluster labels are exported as a BI-ready CSV file and then imported into Power BI.

```
# ============================================================
# 7. BI-READY CUSTOMER RFM TABLE
# ============================================================

fact_rfm = rfm[[
    "Customer.ID",
    "Recency",
    "Frequency",
    "Monetary",
    "RFM_Cluster"
]]

fact_rfm.to_csv(
    r"C:\data\BI_project_superstore\customer_rfm.csv",
    index=False
)
```

Purpose:

To classify customers based on purchasing behavior and enable segment-level analysis.

Integration:

The RFM table is linked to the customer dimension using CustomerID. This allows business users to analyze sales performance, profitability, and customer behavior by RFM cluster directly within the dashboard. This allows business users to analyze sales performance, profitability, and customer behavior by RFM cluster directly within the dashboard.

2.4.2 Machine Learning Output Integration (Shipping Performance & Profit Margin Models)

Machine learning model outputs are also exported from Python and integrated into Power BI as separate analytical tables.

```
# 1. Exporting Predictions and Residuals for the Scatter Plot
# This will allow us to recreate our accuracy plots in Power BI
results_export = X_test_ship.copy()
results_export['Actual_Shipping'] = y_test_ship
results_export['Predicted_Shipping'] = y_pred
results_export['Residual'] = y_test_ship - y_pred
results_export['Abs_Error'] = results_export['Residual'].abs()

results_export.to_csv('C:/Users/User/Desktop/MY Masters(HFU)/Semester_2 Courses/Bi_Big Data Project/XGBoost_Shipping_Performance.csv', index=False)

print("Files exported! we can now import these into Power BI.")
```

```
Files exported! we can now import these into Power BI.
```

1. Shipping Performance (XGBoost Model)

- Contains actual shipping values, predicted shipping values, and residuals.
- Enables evaluation of model accuracy using scatter plots and error analysis in Power BI.

2. **Profit Margin Model Output**
   - Includes predicted margin values and key explanatory features.
   - Supports analysis of profit drivers and comparison between actual and predicted margins.

These outputs are treated as analytical fact tables and connected to the relevant dimensions (e.g., market, product category) to allow interactive exploration.